

SHOPINGY - Vendor Brief: Functional Requirements (Wishlist)

Client: Shopingy s.r.o. | Date: 2026-03-25

Confidential - no NDA in place. Distribute to evaluated vendors only.

1. Project Overview

SHOPINGY is a B2B SaaS platform providing retail market intelligence to shopping mall operators, retail brands, and real estate investors across 40+ countries. The platform manages a database of 340,000+ tenants across 7,700+ shopping malls with ~5 years of historical data.

Project goal: Build a new AI-native frontend application that transforms how users interact with the SHOPINGY database - from static filtering to intelligent, conversational querying with rich analytical outputs.

This is a full-stack build - new frontend, middleware (Querying Catalog), and AI integration layer. The existing backend and database remain unchanged. The new application connects to the existing infrastructure via REST API.

2. Core Problem Statement

Clients currently use only ~10% of the data's potential. The existing interface offers table-based filtering and Excel exports, but cannot:

- Answer natural language questions about the data
- Generate trend analyses or historical comparisons
- Produce executive summaries or visual reports
- Proactively surface insights the user didn't think to ask for

The new product must unlock the remaining 90% of value from the existing dataset.

3. User Cases

Based on the client's feedback - DOCX

3.1 Primary User: Shopping Mall Asset Manager

Scenario: Asset manager of a shopping center wants to benchmark against competitors, track tenant movements, and prepare reports for investors.

Use Case: Mall comparison

Example Query: "Compare OC Chodov and OC Letňany - show tenant changes over the past 6 months"

Expected Output: Executive summary + comparison table + graph

Use Case: Brand tracking

Example Query: "Which malls did H&M exit in the past 12 months across Europe?"

Expected Output: Map/list of exits + timeline

Use Case: Occupancy analysis

Example Query: "Show me the occupancy trend for Polish shopping malls over the last 2 years"

Expected Output: Line chart + data table + CSV export

Use Case: Market summary

Example Query: "Give me an executive summary of the Czech shopping mall market"

Expected Output: Text report with key metrics, top brands, vacancy rates, recent changes

Use Case: Competitive intelligence

Example Query: "Which brands are expanding most aggressively in Nordic shopping malls?"

Expected Output: Ranked list with growth metrics + graph

3.2 Primary User: Retail Brand Expansion Manager

Scenario: Expansion manager at a retail brand evaluating where to open next.

Use Case: Site selection

Example Query: "Show me shopping malls in Germany with vacancy rates above 10%"

Expected Output: Filtered list + map view

Use Case: Competitor mapping

Example Query: "Where is Zara present in Polish malls where we (H&M) are not?"

Expected Output: Gap analysis table + export

Use Case: Trend analysis

Example Query: "Which retail categories are growing in Central European malls?"

Expected Output: Category trend chart + summary

3.3 Primary User: Real Estate Investor

Scenario: Investor evaluating a shopping mall acquisition.

Use Case: Due diligence

Example Query: "Show me the full tenant history of Mall X over the past 3 years"

Expected Output: Timeline + tenant turnover metrics

Use Case: Portfolio comparison

Example Query: "Compare performance of my 5 malls vs. market average"

Expected Output: Benchmark report + CSV

4. Functional Requirements

4.1 User Interface

Requirement: Chat-based query interface

Priority: Must have

Description: Natural language input - users type questions in plain language

Requirement: Conversation history

Priority: Should have

Description: Persistent chat history per session. User can reference previous queries.

Requirement: Fallback to classic view

Priority: Must have

Description: One-click toggle to the existing table/filter interface (Excel-like view)

Requirement: Dashboard view

Priority: Could have

Description: Summary dashboard on login with key metrics and recent activity

Requirement: Multi-language support

Priority: Should have

Description: UI and AI responses in English, Czech, and German at a minimum

Requirement: Platform personalisation

Priority: Should have

Description: Adapted interface/features per client type (mall operator vs. brand vs. investor)

Requirement: Automatic alerts

Priority: Should have

Description: market changes - alerts to email (opening/closing of a store. changes in Malls, changes in geographic locations)

Requirement: Responsive design

Priority: Should have

Description: Desktop (primary)

4.2 AI Agent & Query Processing

Requirement: Natural language querying

Priority: Must have

Description: User types a question and receives an intelligent, data-grounded response

Requirement: Querying Catalogue (middleware)

Priority: Must have

Description: Intermediate layer between frontend and backend. Interprets queries, determines required data subset, fetches only what is needed. AI agent does NOT get direct full-database access.

Requirement: Geographic access enforcement

Priority: Must have

Description: Catalog respects user permissions - user with Poland access cannot query Germany data

Requirement: Data accuracy safeguards

Priority: Must have

Description: AI responses must be grounded in actual data. Hallucination prevention is critical. The system must cite specific data points.

Requirement: Data quality transparency

Priority: Must have

Description: System must clearly communicate where data comes from and how data correctness is ensured

Requirement: GPS/location data handling

Priority: Must have

Description: Handle shops without GPS coordinates gracefully. Use hard-coded keys (e.g., mall ID) for reliable data retrieval rather than relying solely on GPS. Account for exceptions (e.g., all shops in a city assigned to city-level GPS).

Requirement: Context-aware follow-ups

Priority: Should have

Description: User can ask follow-up questions referencing previous results

4.3 Output Types

The AI agent produces multiple output formats - any or all at the same time per query:

Output Type: Executive summary

Description: Natural language text summarising findings and key insights

Output Type: Data table

Description: Structured tabular data showing underlying results

Output Type: CSV/Excel export

Description: Downloadable file with the queried dataset

Output Type: Graphs & charts

Description: Bar charts, line charts, trend visualisations

Output Type: Timeline visualisation

Description: Historical timeline of events (openings, closures, brand entries/exits)

Output Type: Comparison view

Description: Side-by-side comparison of malls, brands, or markets

4.4 Authentication & Access Control

Requirement: User login system

Description: Secure authentication with individual user accounts

Requirement: Country-level permissions

Description: Each user is authorised for specific countries. AI must enforce on all queries.

Requirement: Multi-country access

Description: Some users access all of Europe, others one country. Configurable per user.

Requirement: Admin panel

Description: SHOPINGY team manages users, permissions, and usage analytics

4.5 Credit & Usage System

Requirement: Credit-based query metering

Description: Each AI query costs credits scaled to complexity/data volume

Requirement: Credit balance & tracking

Description: Users see remaining credits and usage history

Requirement: Data extraction protection

Description: Prevent bulk extraction via repeated queries. Rate limiting + credit pricing.

Requirement: Free trial version

Description: Free/limited version of Shoppingy for trial period to onboard new clients **with limited access of queries.**

Requirement: Credit allocation management

Description: Admin allocates credits, sets limits, top-ups

4.6 Brand Functionality

Requirement: Brand-level analytics

Description: Query and analyze at brand/mall level - presence, expansion, exits across markets

4.7 Integration Requirements

Requirement: REST API to the existing backend

Description: New app reads from the existing SHOPPINGY backend via REST endpoints. No direct DB writes.

Requirement: Read-only data access

Description: The AI layer has read-only access to the underlying data

Requirement: Classic view toggle

Description: Seamless switch between the new AI interface and the existing table/filter view

4.8 Security & Data Protection

Requirement: SQL injection prevention

Description: AI-generated queries must be sanitised. No raw SQL passthrough from user input to the database.

Requirement: Data encryption

Description: Encryption at rest and in transit

Requirement: Role-based access control

Description: Permissions enforced at the application and data query level

Requirement: User-side data security

Description: Ensure users cannot access, extract, or manipulate data beyond their permissions

Requirement: Audit logging

Description: Log of all queries, exports, and access events

Requirement: GDPR compliance

Description: EU data protection standards

4.9 Support & Monetization

Requirement: Support service integration

Description: Built-in support/help functionality - potential monetisation channel

5. Non-Functional Requirements

Requirement: Response time (simple queries)

Target: < 10 seconds

Requirement: Response time (complex analysis)

Target: < 30 seconds

Requirement: Availability

Target: 98% uptime

Requirement: Concurrent users

Target: 100+

Requirement: Data freshness

Target: Near-real-time sync with backend (< 1 hour delay)

Requirement: Scalability

Target: Must handle growing database (new malls, countries, users) without redesign

6. Development Approach Expectations

- Speed and adaptability are critical - we prefer an iterative approach:
- Beta version → feedback from first clients → iterations → second version
- No over-engineering of LLM model testing - analysis within business model and pricing scope. Switching LLM models should be straightforward.
- Avoid vendor lock-in - no proprietary platforms with mandatory monthly subscriptions that lock SHOPINGY into a single provider
- Solid foundation first - the underlying data product and access model must work reliably before layering AI complexity on top.

7. Deliverables Expected from Vendor

1. Technical proposal - architecture, technology choices, integration approach
2. Timeline - phased delivery plan with milestones
3. Cost breakdown - itemized (development, infrastructure, etc.)
4. Team composition - who works on the project, roles, experience
5. Maintenance & support plan - post-launch SLA, updates
6. No vendor lock-in statement - confirm the solution is portable and not tied to proprietary platforms

For additional questions, please contact:

Martin Landsmann, AI Product Manager (martin@rousing-ai.com)

Hana Kabourková, Global Business Development Director (kabourkova@shoppingy.com)